Introduction to Coroutines in Android Studio

June 16, 2026 5:00 PM

https://developer.android.com/codelabs/basic-android-kotlin-compose-coroutines-android-studio#0

Introduction to Coroutines in Android Studio

About this codelab

Last updated May 21, 2024

Written by Google Developers Training team

1. Before you begin

In the previous codelab, you learned about coroutines. You used Kotlin Playground to write concurrent code using coroutines. In this codelab, you'll apply your knowledge of coroutines within an Android app and its lifecycle. You'll add code to launch new coroutines concurrently and learn how to test them.

Prerequisites

- Knowledge of Kotlin language basics, including functions and lambdas
- Able to build layouts in Jetpack Compose
- Able to write unit tests in Kotlin (refer to Write unit tests for ViewModel codelab)
- How threads and concurrency work
- Basic knowledge of coroutines and CoroutineScope

What you'll build

- Race Tracker app that simulates race progress between two players. Consider this app as a chance to experiment and learn more about different aspects of coroutines.

What you'll learn

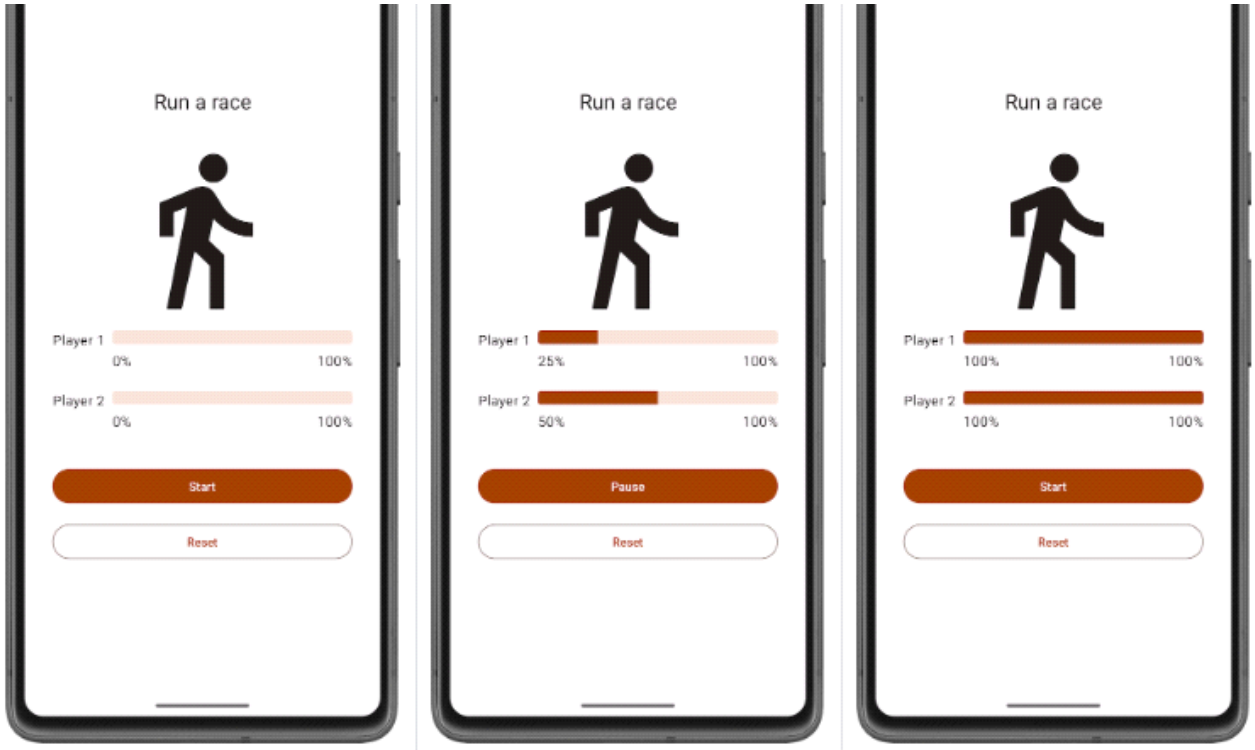
- ✓ Using coroutines in Android app lifecycle.
- ✓ The principles of structured concurrency.
- ✓ How to write unit tests to test the coroutines.

What you'll need

- The latest stable version of Android Studio

2. App overview

The Race Tracker app simulates two players running a race. The app UI consists of two buttons, **Start/Pause** and **Reset**, and two progress bars to show the progress of the racers. Players 1 and 2 are set to "run" the race at different speeds. When the race starts Player 2 progresses twice as fast as Player 1.



You will use coroutines in the app to ensure:

- Both players "run the race" concurrently.
- The app UI is responsive and the progress bars increments during the race.

The starter code has the UI code ready for the Race Tracker app. The main focus of this part of the codelab is to get you familiar with Kotlin coroutines inside an Android app.

Get the starter code

To get started, download the starter code:

Download zip

Alternatively, you can clone the GitHub repository for the code:

```
$ git clone https://github.com/google-developer-training/basic-android-kotlin-compose-tr
$ cd basic-android-kotlin-compose-training-race-tracker
$ git checkout starter
```

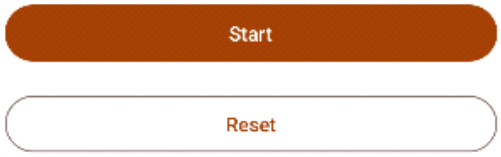
Note: The starter code is in the `starter` branch of the downloaded repository.

You can browse the starter code in the Race Tracker GitHub repository.

Starter code walkthrough

You can start the race by clicking the **Start** button. The text of the **Start** button changes to **Pause** while the race is in progress.

You can start the race by clicking the **Start** button. The text of the **Start** button changes to **Pause** while the race is in progress.



At any point in time, you can use this button to pause or continue the race.

When the race starts, you can see the progress of each player through a status indicator. The `StatusIndicator` composable function displays the progress status of each player. It uses the `LinearProgressIndicator` composable to display the progress bar. You'll be using coroutines to update the value for progress.

`RaceParticipant` provides the data for progress increment. This class is a state holder for each of the players and maintains the `name` of the participant, the `maxProgress` to reach to finish the race, the delay duration between progress increments, `currentProgress` in race and the `initialProgress`.

In the next section, you will use coroutines to implement the functionality to simulate the race progress without blocking the app UI.

3. Implement race progress

You need the `run()` function which compares the player's `currentProgress` with the `maxProgress`, which reflects the total progress of the race, and uses the `delay()` suspend function to add a slight delay between progress increments. This function must be a `suspend` function since it is calling another suspend function `delay()`. Also, you will call this function later in the codelab from a coroutine. Follow these steps to implement the function:

1. Open `RaceParticipant` class, which is part of the starter code.
2. Inside the `RaceParticipant` class, define a new `suspend` function named `run()`.

```
class RaceParticipant(
    ...
) {
    var currentProgress by mutableStateOf(initialProgress)
        private set
```

```
    var currentProgress by mutableStateOf(initialProgress)
        private set

    suspend fun run() {

    }
    ...
}
```

3. To simulate the progress of the race, add a `while` loop that runs until `currentProgress` reaches the value `maxProgress`, which is set to `100`.

```
class RaceParticipant(
    ...
    val maxProgress: Int = 100,
    ...
) {
    var currentProgress by mutableStateOf(initialProgress)
        private set

    suspend fun run() {
        while (currentProgress < maxProgress) {

        }
    }
    ...
}
```

4. The value of the `currentProgress` is set to `initialProgress`, which is `0`. To simulate the participant's progress, increment the value `currentProgress` by the value of `progressIncrement` property inside the while loop. Note that the default value of `progressIncrement` is `1`.

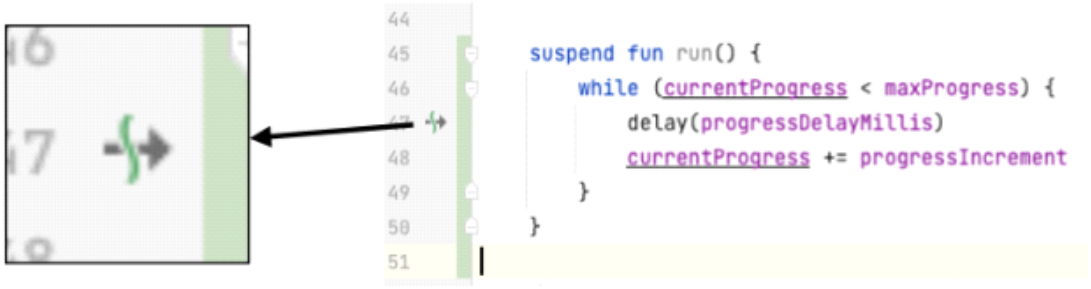
```
class RaceParticipant(
    ...
    val maxProgress: Int = 100,
    ...
    private val progressIncrement: Int = 1,
    private val initialProgress: Int = 0
) {
    ...
    var currentProgress by mutableStateOf(initialProgress)
        private set

    suspend fun run() {
        while (currentProgress < maxProgress) {
            currentProgress += progressIncrement
        }
    }
}
```

5. To simulate different progress intervals in the race, use the `delay()` suspend function. Pass the value of the `progressDelayMillis` property as an argument

5. To simulate different progress intervals in the race, use the `delay()` suspend function. Pass the value of the `progressDelayMillis` property as an argument.

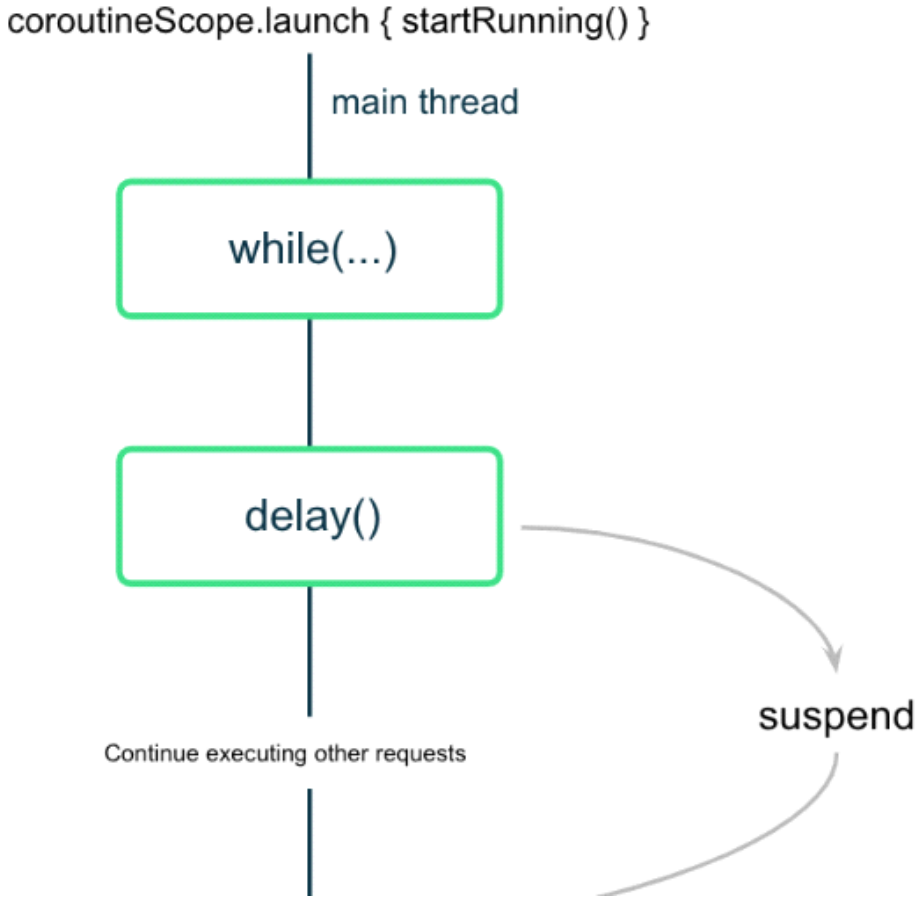
```
suspend fun run() {
    while (currentProgress < maxProgress) {
        delay(progressDelayMillis)
        currentProgress += progressIncrement
    }
}
```

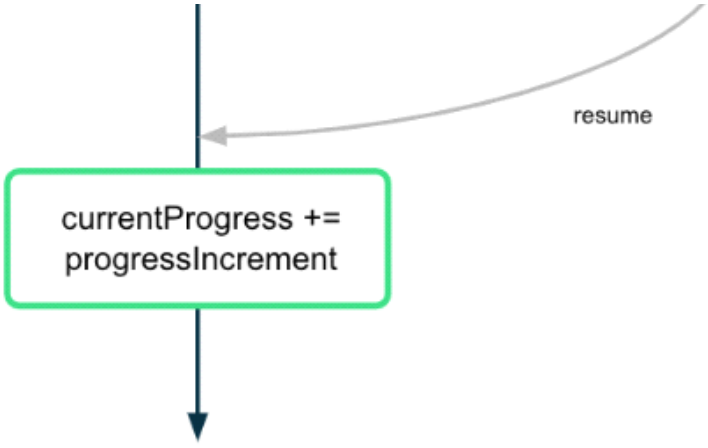
When you look at the code you just added, you will see an icon on the left of the call to the `delay()` function in Android Studio, as shown in the screenshot below:



This icon indicates the suspension point when the function might suspend and resume again later.

The main thread is not blocked while the coroutine is waiting to complete the delay duration, as shown in the following diagram:





The coroutine suspends (but doesn't block) the execution after calling the `delay()` function with the desired interval value. Once the delay is complete, the coroutine resumes the execution and updates the value of the `currentProgress` property.

4. Start the race

When the user presses the **Start** button, you need to "start the race" by calling the `run()` suspend function on each of the two player instances. To do this, you need to launch a coroutine to call the `run()` function.

When you launch a coroutine to trigger the race, you need to ensure the following aspects for both participants:

- They start running as soon as the **Start** button is clicked—that is, the coroutines launch.
- They pause or stop running when the **Pause** or **Reset** button is clicked, respectively—that is, the coroutines are canceled.
- When the user closes the app, the cancellation is properly managed—that is, all the coroutines are canceled and bound to a lifecycle.

In the first codelab you learned that you can only call a suspend function from another suspend function. To call suspend functions safely from inside a composable, you need to use the `LaunchedEffect()` composable. `LaunchedEffect()` composable runs the provided suspending function for as long as it remains in the composition. You can use the `LaunchedEffect()` composable function to accomplish all of the following:

- The `LaunchedEffect()` composable allows you to safely call suspend functions from composables.
- When the `LaunchedEffect()` function enters the Composition, it launches a coroutine with the code block passed as a parameter. It runs the provided suspend function as long as it remains in the composition. When a user clicks the **Start** button in the RaceTracker app, the `LaunchedEffect()` enters the composition and launches a coroutine to update progress.
- The coroutine is canceled when the `LaunchedEffect()` exits the composition. In the app, if the user clicks the **Reset**/**Pause** button, `LaunchedEffect()` is removed from the composition and the underlying coroutines are canceled.

For the RaceTracker app, you don't have to provide a Dispatcher explicitly, since `LaunchedEffect()` takes care of it.

To start the race call the `run()` function for each participant and perform the following steps:

1. Open the `RaceTrackerApp.kt` file located in the `com.example.racetracker.ui` package.
2. Navigate to `RaceTrackerApp()` composable and add a call to the `LaunchedEffect()` composable on the line

1. Open the `RaceTrackerApp.kt` file located in the `com.example.racetracker.ui` package.
2. Navigate to `RaceTrackerApp()` composable and add a call to the `LaunchedEffect()` composable on the line after the definition of `raceInProgress`.

```
@Composable
fun RaceTrackerApp() {
    ...
    var raceInProgress by remember { mutableStateOf(false) }

    LaunchedEffect {

    }
    RaceTrackerScreen(...)
}
```

3. To ensure that if the instances of `playerOne` or `playerTwo` are replaced with different instances, then `LaunchedEffect()` needs to cancel and relaunch the underlying coroutines, add the `playerOne` and `playerTwo` objects as `key` to the `LaunchedEffect`. Similar to how a `Text()` composable gets recomposed when its text value changes, if any of the key arguments of the `LaunchedEffect()` changes, the underlying coroutine is canceled and relaunched.

```
LaunchedEffect(playerOne, playerTwo) {
}
```

4. Add a call to the `playerOne.run()` and `playerTwo.run()` functions.

```
@Composable
fun RaceTrackerApp() {
    ...
    var raceInProgress by remember { mutableStateOf(false) }

    LaunchedEffect(playerOne, playerTwo) {
        playerOne.run()
        playerTwo.run()
    }
    RaceTrackerScreen(...)
}
```

5. Wrap the `LaunchedEffect()` block with an `if` condition. The initial value for this state is `false`. The value for the `raceInProgress` state is updated to `true` when the user clicks the **Start** button and the `LaunchedEffect()` executes.

```
if (raceInProgress) {
    LaunchedEffect(playerOne, playerTwo) {
        playerOne.run()
        playerTwo.run()
    }
}
```

```
        playerTwo.run()
    }
}
```

6. Update the `raceInProgress` flag to `false` to finish the race. This value is set to `false` when the user clicks on **Pause** too. When this value is set to `false` the `LaunchedEffect()` ensures that all the launched coroutines are canceled.

```
LaunchedEffect(playerOne, playerTwo) {
    playerOne.run()
    playerTwo.run()
    raceInProgress = false
}
```

7. Run the app and click **Start**. You should see player one completes the race before player two starts running, as shown in the following video:

This doesn't look like a fair race! In the next section, you will learn how to launch concurrent tasks so that both players can run at the same time, understand the concepts, and implement this behavior.

5. Structured concurrency

The way you write code using coroutines is called structured concurrency. This style of programming improves the readability and development time of your code. The idea of structured concurrency is that coroutines have a hierarchy—tasks might launch subtasks, which might launch subtasks in turn. The unit of this hierarchy is referred to as a coroutine scope. Coroutine scopes should always be associated with a lifecycle.

The Coroutines APIs adhere to this structured concurrency by design. You cannot call a suspend function from a function which is not marked suspend. This limitation ensures that you call the suspend functions from coroutine builders, such as `launch`. These builders are, in turn, tied to a `CoroutineScope`.

6. Launch concurrent tasks

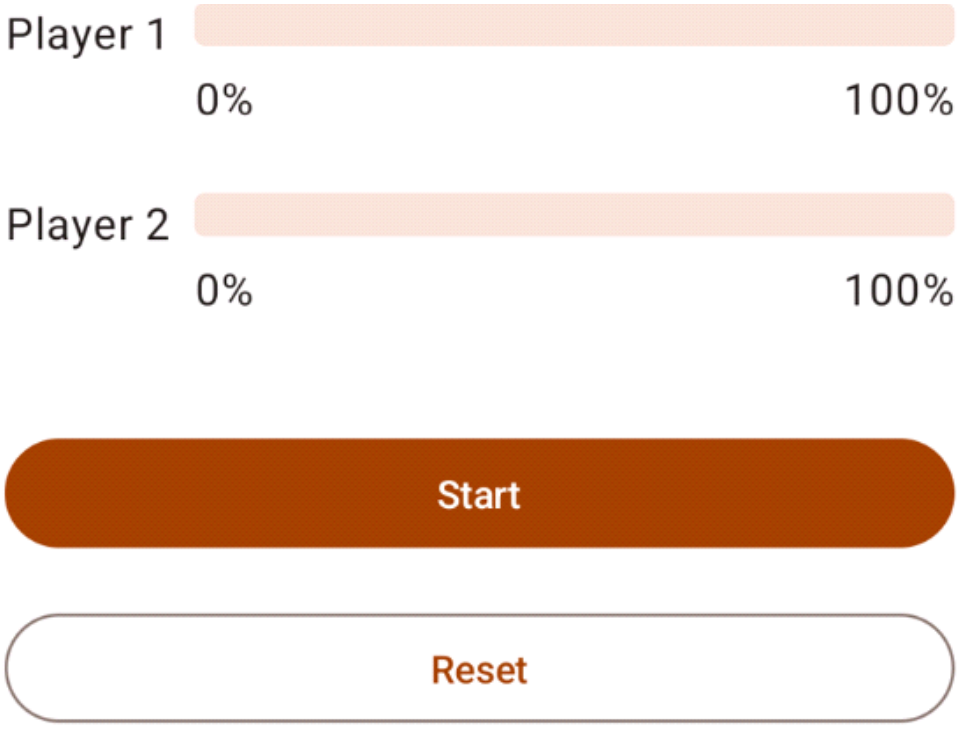
1. To allow both participants to run concurrently, you need to launch two separate coroutines and move each call to the `run()` function inside those coroutines. Wrap the call to the `playerOne.run()` with `launch` builder.

```
LaunchedEffect(playerOne, playerTwo) {
    launch { playerOne.run() }
    playerTwo.run()
    raceInProgress = false
}
```

2. Similarly, wrap the call to the `playerTwo.run()` function with the `launch` builder. With this change, the app launches two coroutines that execute concurrently. Both players can now run at the same time.

```
LaunchedEffect(playerOne, playerTwo) {
    launch { playerOne.run() }
    launch { playerTwo.run() }
    raceInProgress = false
}
```

3. Run the app and click **Start**. While you expect the race to start, the text of the button immediately changes back to **Start** unexpectedly.



When both the players complete their run, the Race Tracker app should reset the text for the **Pause** button back to **Start**. However, right now the app updates the `raceInProgress` right away after launching the coroutines without waiting the players to complete the race:

```
LaunchedEffect(playerOne, playerTwo) {
    launch {playerOne.run() }
    launch {playerTwo.run() }
    raceInProgress = false // This will update the state immediately, without waiting fo
}
```

The `raceInProgress` flag is updated immediately because:

- The `launch` builder function launches a coroutine to execute `playerOne.run()` and immediately returns to execute the next line in the code block.
- The same execution flow happens with the second `launch` builder function that executes `playerTwo.run()` function.
- As soon as the second `launch` builder returns, the `raceInProgress` flag is updated. This immediately changes the button text to **Start** and the race does not begin.

Coroutine Scope

The `coroutineScope` suspend function creates a `CoroutineScope` and calls the specified suspend block with the current scope. The scope inherits its `coroutineContext` from the `LaunchedEffect()` scope.

The scope returns as soon as the given block and all its children coroutines are complete. For the `RaceTracker` app, it returns once both participant objects finish executing the `run()` function.

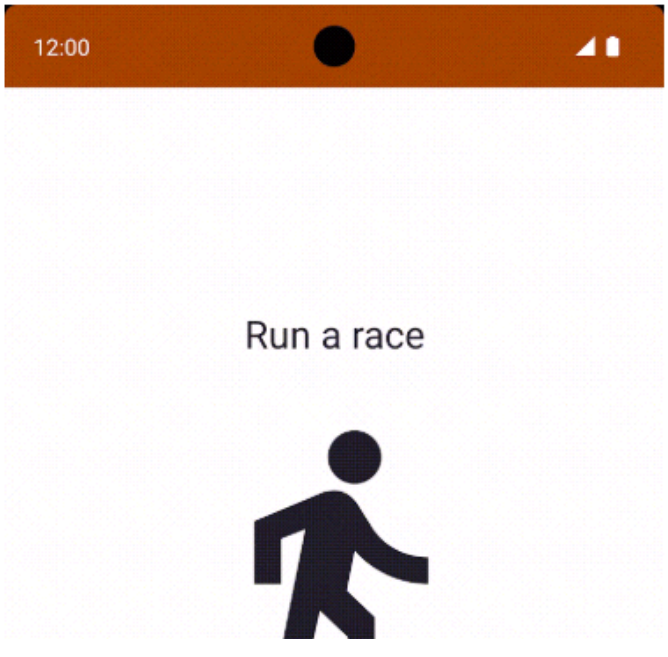
1. To ensure the `run()` function of `playerOne` and `playerTwo` completes execution before updating the `raceInProgress` flag, wrap both launch builders with a `coroutineScope` block.

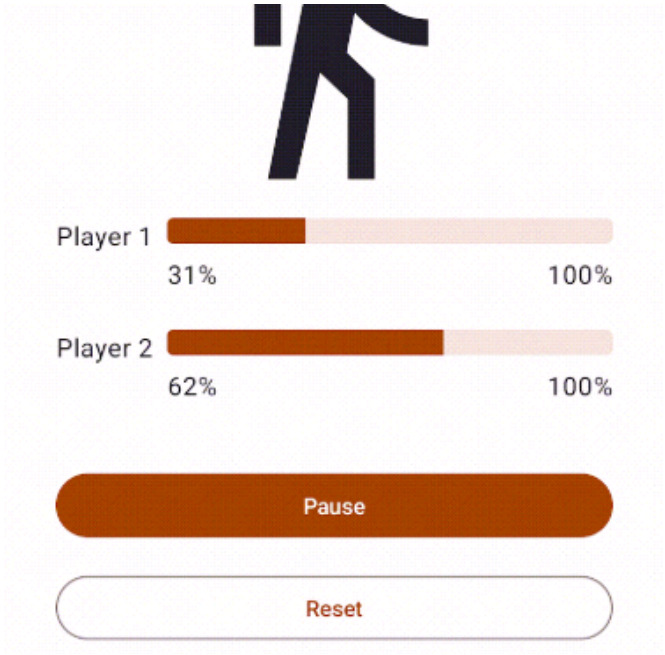
`raceInProgress` flag, wrap both launch builders with a `coroutineScope` block.

```
LaunchedEffect(playerOne, playerTwo) {
    coroutineScope {
        launch { playerOne.run() }
        launch { playerTwo.run() }
    }
    raceInProgress = false
}
```

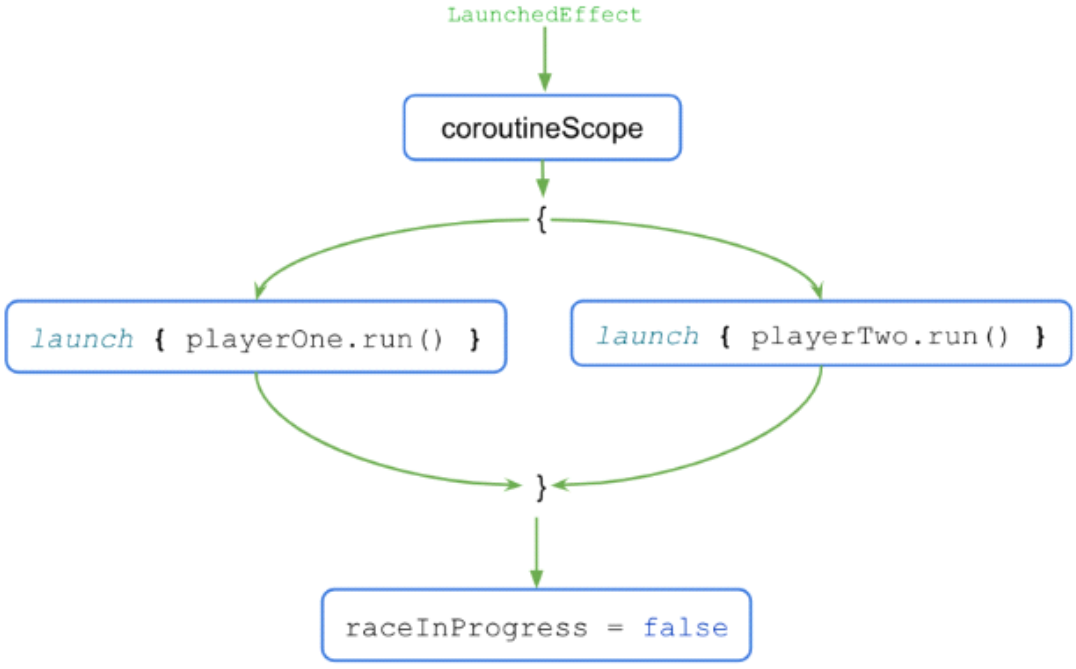
2. Run the app on an emulator/Android device. You should see following screen:

3. Click the **Start** button. Player 2 runs faster than Player 1. After the race is complete, which is when both players reach 100% progress, the label for the **Pause** button changes to **Start**. You can click the **Reset** button to reset the race and re-execute the simulation. The race is shown in the following video.





The execution flow is shown in the following diagram:



- When the `LaunchedEffect()` block executes, the control is transferred to the `coroutineScope{..}` block.
- The `coroutineScope` block launches both coroutines concurrently and waits for them to finish execution.
- Once the execution is complete, the `raceInProgress` flag updates.

The `coroutineScope` block only returns and moves on after all the code inside the block completes execution. For the code outside of the block, the presence or absence of concurrency becomes a mere implementation detail. This coding style provides a structured approach to concurrent programming and is referred to as structured concurrency.

When you click the **Reset** button after the race completes, the coroutines are canceled, and the progress for both players is reset to `0`.

To see how coroutines are canceled when the user clicks the **Reset** button, follow these steps:

1. Wrap the body of the `run()` method in a try-catch block as shown in the following code:

1. Wrap the body of the `run()` method in a try-catch block as shown in the following code:

```
suspend fun run() {
    try {
        while (currentProgress < maxProgress) {
            delay(progressDelayMillis)
            currentProgress += progressIncrement
        }
    } catch (e: CancellationException) {
        Log.e("RaceParticipant", "$name: ${e.message}")
        throw e // Always re-throw CancellationException.
    }
}
```

2. Run the app and click the **Start** button.
3. After some progress increments, click the **Reset** button.
4. Make sure you see the following message printed in Logcat:

```
Player 1: StandaloneCoroutine was cancelled
Player 2: StandaloneCoroutine was cancelled
```

7. Write unit tests to test coroutines

Unit testing code that uses coroutines requires some extra attention, as their execution can be asynchronous and happen across multiple threads.

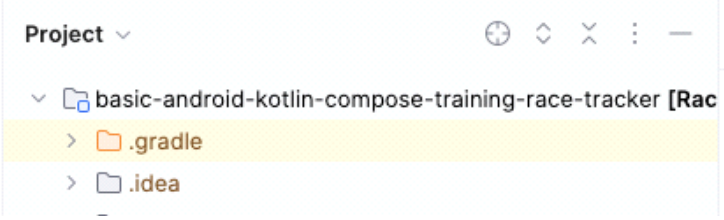
To call suspending functions in tests, you need to be in a coroutine. As JUnit test functions themselves aren't suspending functions, you need to use the `runTest` coroutine builder. This builder is part of the `kotlinx-coroutines-test` library and is designed to execute tests. The builder executes the test body in a new coroutine.

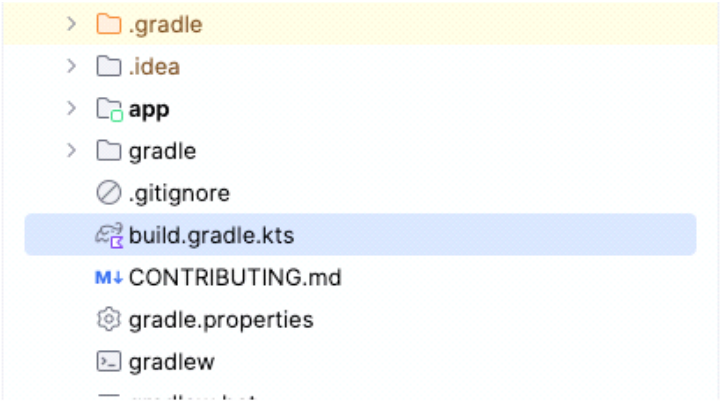
> **Note**: Coroutines can be started not only directly in the test body, but also by the objects being used in the test using `runTest`.

Since `runTest` is part of the `kotlinx-coroutines-test` library, you need to add its dependency.

To add the dependency, complete the following steps:

1. Open the app module's `build.gradle.kts` file, located in the `app` directory in the **Project** pane.

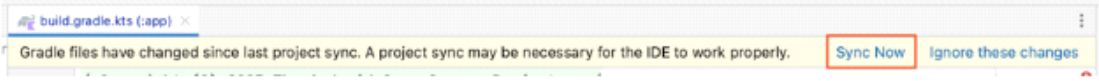




2. Inside the file, scroll down until you find the `dependencies{}` block.
3. Add a dependency using the `testImplementation` config to the `kotlinx-coroutines-test` library.

```
plugins {
    ...
}

android {
    ...
}

dependencies {
    ...
    testImplementation("org.jetbrains.kotlinx:kotlinx-coroutines-test:1.6.4")
}
```

4. In the notification bar at the top of the **build.gradle.kts** file, click **Sync Now** to let the import and build finish, as shown in the following screenshot:



Once the build is complete, you can start writing tests.

Implement unit tests for starting and finishing the race

To ensure the race progress updates correctly during different phases of the race, your unit tests need to cover different scenarios. For this codelab, two scenarios are covered:

- Progress after the race starts.
- Progress after the race finishes.

To check if the race progress updates correctly after the start of the race, you need to assert that the current progress is set to 1 after the `raceParticipant.progressDelayMillis` duration is passed.

To implement the test scenario, follow these steps:

1. Navigate to the `RaceParticipantTest.kt` file located under the test source set.
2. To define the test, after the `raceParticipant` definition, create a `raceParticipant_RaceStarted_ProgressUpdated()` function and annotate it with the `@Test` annotation. Since the test block needs to be placed in the `runTest` builder, use the expression syntax to return the

2. To define the test, after the `raceParticipant` definition, create a `raceParticipant_RaceStarted_ProgressUpdated()` function and annotate it with the `@Test` annotation. Since the test block needs to be placed in the `runTest` builder, use the expression syntax to return the `runTest()` block as a test result.

```
class RaceParticipantTest {
    private val raceParticipant = RaceParticipant(
        ...
    )

    @Test
    fun raceParticipant_RaceStarted_ProgressUpdated() = runTest {
    }
}
```

3. Add a read-only `expectedProgress` variable and set it to `1`.

```
@Test
fun raceParticipant_RaceStarted_ProgressUpdated() = runTest {
    val expectedProgress = 1
}
```

4. To simulate the race start, use the `launch` builder to launch a new coroutine and call the `raceParticipant.run()` function.

```
@Test
fun raceParticipant_RaceStarted_ProgressUpdated() = runTest {
    val expectedProgress = 1
    launch { raceParticipant.run() }
}
```

> **Note:** You can directly call the `raceParticipant.run()` in the `runtTest` builder, but the default test implementation ignores the call to `delay()`. As a result, the `run()` finishes executing before you can analyze the progress.

The value of the `raceParticipant.progressDelayMillis` property determines the duration after which the race progress updates. In order to test the progress after `progressDelayMillis` time has elapsed, you need to add some form of delay to your test.

5. Use the `advanceTimeBy()` helper function to advance the time by the value of `raceParticipant.progressDelayMillis`. The `advanceTimeBy()` function helps to reduce the test execution time.

```
@Test
fun raceParticipant_RaceStarted_ProgressUpdated() = runTest {
    val expectedProgress = 1
    launch { raceParticipant.run() }
```

```
fun raceParticipant_RaceStarted_ProgressUpdated() = runTest {
    val expectedProgress = 1
    launch { raceParticipant.run() }
    advanceTimeBy(raceParticipant.progressDelayMillis)
}
```

6. Since `advanceTimeBy()` doesn't run the task scheduled at the given duration, you need to call the `runCurrent()` function. This function executes any pending tasks at the current time.

```
@Test
fun raceParticipant_RaceStarted_ProgressUpdated() = runTest {
    val expectedProgress = 1
    launch { raceParticipant.run() }
    advanceTimeBy(raceParticipant.progressDelayMillis)
    runCurrent()
}
```

7. To ensure the progress updates, add a call to the `assertEquals()` function to check if the value of the `raceParticipant.currentProgress` property matches the value of the `expectedProgress` variable.

```
@Test
fun raceParticipant_RaceStarted_ProgressUpdated() = runTest {
    val expectedProgress = 1
    launch { raceParticipant.run() }
    advanceTimeBy(raceParticipant.progressDelayMillis)
    runCurrent()
    assertEquals(expectedProgress, raceParticipant.currentProgress)
}
```

8. Run the test to confirm that it passes.

To check if the race progress updates correctly after the race finishes, you need to assert that when the race finishes, the current progress is set to `100`.

Follow these steps to implement the test:

1. After the `raceParticipant_RaceStarted_ProgressUpdated()` test function, create a `raceParticipant_RaceFinished_ProgressUpdated()` function and annotate it with the `@Test` annotation. The function should return a test result from the `runTest{}` block.

```
class RaceParticipantTest {
    ...

    @Test
    fun raceParticipant_RaceStarted_ProgressUpdated() = runTest {
        ...
    }

    @Test
    fun raceParticipant_RaceFinished_ProgressUpdated() = runTest {
    }
```

```
    @Test
    fun raceParticipant_RaceFinished_ProgressUpdated() = runTest {
    }
}
```

2. Use `launch` builder to launch a new coroutine and add a call to the `raceParticipant.run()` function in it.

```
@Test
fun raceParticipant_RaceFinished_ProgressUpdated() = runTest {
    launch { raceParticipant.run() }
}
```

3. To simulate the race finish, use the `advanceTimeBy()` function to advance the dispatcher's time by `raceParticipant.maxProgress * raceParticipant.progressDelayMillis`:

```
@Test
fun raceParticipant_RaceFinished_ProgressUpdated() = runTest {
    launch { raceParticipant.run() }
    advanceTimeBy(raceParticipant.maxProgress * raceParticipant.progressDelayMillis)
}
```

4. Add a call to the `runCurrent()` function to execute any pending tasks.

```
@Test
fun raceParticipant_RaceFinished_ProgressUpdated() = runTest {
    launch { raceParticipant.run() }
    advanceTimeBy(raceParticipant.maxProgress * raceParticipant.progressDelayMillis)
    runCurrent()
}
```

5. To ensure the progress updates correctly, add a call to the `assertEquals()` function to check if the value of the `raceParticipant.currentProgress` property is equal to `100`.

```
@Test
fun raceParticipant_RaceFinished_ProgressUpdated() = runTest {
    launch { raceParticipant.run() }
    advanceTimeBy(raceParticipant.maxProgress * raceParticipant.progressDelayMillis)
    runCurrent()
    assertEquals(100, raceParticipant.currentProgress)
}
```

6. Run the test to confirm that it passes.

Try this challenge

Apply the test strategies discussed in the Write unit tests for ViewModel codelab. Add the tests to cover the happy path, error cases, and boundary cases.

Apply the test strategies discussed in the Write unit tests for ViewModel codelab. Add the tests to cover the happy path, error cases, and boundary cases.

Compare the test you write with the ones available in the solution code.

8. Get the solution code

To download the code for the finished codelab, you can use these git commands:

```
git clone https://github.com/google-developer-training/basic-android-kotlin-compose-trai
cd basic-android-kotlin-compose-training-race-tracker
```

Alternatively, you can download the repository as a zip file, unzip it, and open it in Android Studio.

Download zip

basic-and...

Note: The solution code is in the `main` branch of the code repository.

If you want to see the solution code, view it on GitHub.

git clone https://github.com/google-developer-training/basic-android-kotlin-compose-training-race-tracker.git
cd basic-android-kotlin-compose-training-race-tracker

9. Conclusion

Congratulations! You just learned how to use coroutines to handle concurrency. Coroutines help manage long-running tasks that might otherwise block the main thread and cause your app to become unresponsive. You also learned how to write unit tests to test the coroutines.

The following features are some of the benefits of coroutines:

- **Readability:** The code you write with coroutines provides a clear understanding of the sequence that executes the lines of code.
- **Jetpack integration:** Many Jetpack libraries, such as Compose and ViewModel, include extensions that provide full coroutines support. Some libraries also provide their own coroutine scope that you can use for structured concurrency.
- **Structured concurrency:** Coroutines make concurrent code safe and easy to implement, eliminate unnecessary boilerplate code, and ensure that coroutines launched by the app are not lost or keep wasting resources.

Summary

- Coroutines enable you to write long running code that runs concurrently without learning a new style of

Summary

- Coroutines enable you to write long running code that runs concurrently without learning a new style of programming. The execution of a coroutine is sequential by design.
- The `suspend` keyword is used to mark a function, or function type, to indicate its availability to execute, pause, and resume a set of code instructions.
- A `suspend` function can be called only from another suspend function.
- You can start a new coroutine using the `launch` or `async` builder function.
- Coroutine context, coroutine builders, Job, coroutine scope and Dispatcher are the major components for implementing coroutines.
- Coroutines use dispatchers to determine the threads to use for its execution.
- Job plays an important role to ensure structured concurrency by managing the lifecycle of coroutines and maintaining the parent-child relationship.
- A `CoroutineContext` defines the behavior of a coroutine using Job and a coroutine dispatcher.
- A `CoroutineScope` controls the lifetime of coroutines through its Job and enforces cancellation and other rules to its children and their children recursively.
- Launch, completion, cancellation, and failure are four common operations in the coroutine's execution.
- Coroutines follow a principle of structured concurrency.

Learn more

- Kotlin coroutines on Android
- Additional resources for Kotlin coroutines and flow
- Exceptions in Coroutines
- Coroutines on Android (part 1)
- Kotlin coroutines 101